Python from Zero

The concepts you should know. An offline guide.

This book is the concept map of Python. It assumes you can read a sentence and you have not memorized the language. Every idea is defined when it appears. The examples are Python 3.10 or newer. On this machine the installed interpreter is Python 3.14, which runs all of them.

Run a file with `python file.py`. Try a line in the prompt with `python`, then type `exit()` to leave. A concept is yours when you can predict the output before you run it.

Contents

1. Programs, indentation, and comments
2. Objects, names, and mutability
3. None, Booleans, and numbers
4. Text and bytes
5. Operators and truth
6. Choices and loops
7. Functions and scope
8. Lists, tuples, dicts, and sets
9. Comprehensions and unpacking
10. Files and paths
11. Errors
12. Classes
13. The data model
14. Iterators, generators, and with
15. Decorators
16. Modules, packages, and installs
17. Type hints
18. The standard library you actually use
19. Concurrency
20. Tests, debugging, and style
21. Pitfalls
22. Glossary and a study plan
23. Interview questions

1. Programs, indentation, and comments

A Python program is a text file of statements. The interpreter reads the file from top to bottom and runs each statement. Indentation is syntax. A block is the lines indented further than the line that opened them. Use four spaces. Do not mix tabs and spaces.

```
def greet(name):
    if name:
        print("Hello", name)
    else:
        print("Hello")
```

A comment starts with `#` and runs to the end of the line. The interpreter skips it. A docstring is a string sitting as the first statement of a module, function, or class. Tools and `help()` read it. Use a docstring to say what the function promises, not to narrate each line.

```
def area(width, height):
    """Return the area of a rectangle. width and height are numbers."""
    return width * height
```

`print` writes a line of text. It is how you look at a value while learning. In a real program, lasting messages go through the `logging` module, covered later.

2. Objects, names, and mutability

Everything in Python is an **object**: a thing with a type, an identity, and a value. The number `3`, the text `"hi"`, a list, and a function are all objects.

A **name** is a label stuck on an object. Assignment sticks the label on.

```
count = 1
count = count + 1
```

The second line builds a new integer and moves the label `count` onto it. It does not change the object `1`. Integers cannot change. They are **immutable**.

A list can change. It is **mutable**.

```
nums = [1, 2]
also = nums
also.append(3)
# nums and also are the same list: [1, 2, 3]
```

`also = nums` does not copy the list. Both names point at one object. `append` changes that object, so both names see the change. This is the fact behind most “why did my other variable change?” bugs.

`==` asks whether two values compare equal. `is` asks whether two names point at the exact same object. Use `is` for `None` and for this identity question. Use `==` for values.

```
a = [1, 2]
b = [1, 2]
a == b      # True, same contents
a is b      # False, two lists
```

A **shallow copy** makes a new outer container and keeps the same inner objects. A **deep copy** copies the inners too. `list(old)`, `old[:]`, and `copy.copy` are shallow. `copy.deepcopy` walks the whole structure.

```
import copy
grid = [[1], [2]]
shallow = copy.copy(grid)
shallow[0].append(9)
# grid is now [[1, 9], [2]] because the inner list was shared
deep = copy.deepcopy(grid)
deep[0].append(8)
# grid is unchanged by this append
```

3. None, Booleans, and numbers

`None` is the single object that means “no value.” There is only one of it. Test with `result is None`.

Booleans are `True` and `False`. They are the two values of type `bool`.

Integers (`int`) have no fixed size. `2 ** 1000` is a real integer, not an overflow. Floats (`float`) are binary fractions with a limited number of bits. `0.1 + 0.2` is not exactly `0.3`. For money and other exact decimals, use `decimal.Decimal`, not `float`.

```
from decimal import Decimal
price = Decimal("19.99")
tax = Decimal("0.08")
total = price * (1 + tax)
```

Write the decimal as a string. `Decimal(0.1)` starts from the already-inexact float.

Division with `/` always returns a float. `//` is floor division, toward negative infinity. `%` is the remainder. `divmod(a, b)` returns both. `**` is power.

`round` uses banker’s rounding on ties: `round(2.5)` is `2` and `round(3.5)` is `4`, rounding to the even integer. Do not use it as a money policy. Quantize a `Decimal` instead.

4. Text and bytes

A `str` is Unicode text. A `bytes` object is raw octets. Text becomes bytes through an encoding, usually UTF-8.

```
text = "café"
raw = text.encode("utf-8")   # bytes
back = raw.decode("utf-8")    # str
```

Strings are immutable. Methods such as `upper` and `replace` return a new string. Useful tests: `startswith`, `endswith`, `isdigit`, `strip`. `split` breaks a string into a list. `join` builds a string from pieces and is the right way to combine many fragments.

```
words = ["system", "design"]
line = " ".join(words)        # "system design"
```

Indexing starts at 0. A negative index counts from the end. `s[-1]` is the last character. A slice `s[start:stop]` includes `start` and excludes `stop`. `s[::2]` takes every second character. Slicing a string returns a new string.

Formatting: an f-string evaluates expressions inside braces.

```
name = "Priya"
score = 9
label = f"{name} scored {score}"
```

Put a format spec after a colon: `f"{price:.2f}"`. For a value that might contain braces or comes from a template file, `template.format(name=name)` is the other tool. Do not build SQL or HTML by gluing raw user text into a string.

A raw string, `r"\n"`, treats backslashes as ordinary characters. Regular expressions want raw strings so `\d` stays `\d`.

5. Operators and truth

Comparisons chain: `0 <= n < 10` means `0 <= n and n < 10`, and `n` is evaluated once.

`and`, `or`, and `not` combine conditions. `and` and `or` return one of their operands, not always a Boolean.

```
name = "" or "guest"     # "guest"
found = items and items[0]
```

Many values count as false in an `if`: `None`, `False`, `0`, `0.0`, empty text, empty bytes, and empty containers. Everything else counts as true. If you mean "is this missing," write `value is None`. An empty list is false and is not `None`.

Membership is `in`. For a dict, `in` tests keys. For a string, it tests a substring. For a set or dict, the test is fast. For a list, the test walks the list.

The walrus operator `:=` assigns inside an expression.

```
if (line := input()) != "quit":
    print(line)
```

Use it when a name would otherwise be assigned on the line above only so the next line can test it.

6. Choices and loops

```
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
else:
    grade = "C"
```

`if` runs its block when the condition is true. `elif` is another condition, tried only if the earlier ones were false. `else` runs when none matched. One of these blocks runs.

`match` compares a value against patterns. It fits when one value has several shapes.

```
def describe(value):
    match value:
        case 0:
            return "zero"
        case [x, y]:
            return f"pair {x}, {y}"
        case {"code": code}:
            return f"code {code}"
        case _:
            return "other"
```

A `for` loop asks an iterable for items, one at a time.

```
for item in ["a", "b"]:
    print(item)
```

`range(stop)` yields integers from 0 up to but not including `stop`. `range(start, stop, step)` is the full form. `range` does not build a giant list.

`while` repeats as long as a condition stays true. If the condition never becomes false, the loop never ends. That is an infinite loop. Put a change inside the body that moves toward the end.

`break` leaves the loop now. `continue` skips the rest of this round and starts the next. A loop may have an `else`. That `else` runs when the loop finished without `break`. It is useful for search: “we walked the whole list and did not find it.”

```
for user in users:
    if user.name == "Priya":
        break
else:
    user = None
```

7. Functions and scope

A function packages steps under a name. `return` sends a value back and ends the function. A function with no `return` returns `None`.

```
def clamp(n, low, high):
    if n < low:
        return low
    if n > high:
        return high
    return n
```

Parameters can have defaults. The default is computed once, when the function is defined, not on each call. A mutable default is shared by every caller that omits the argument. That is a classic bug.

```
def add_item(item, bucket=None):
    if bucket is None:
        bucket = []
    bucket.append(item)
    return bucket
```

Arguments can be passed by position or by name. A `/` in the parameter list means “everything before me is positional only.” A `*` means “everything after me is keyword only.”

```
def build(name, /, size, *, bold=False):
    return name, size, bold

build("title", 12, bold=True)
```

`*args` collects extra positional arguments into a tuple. `**kwargs` collects extra keyword arguments into a dict.

```
def show(*args, **kwargs):
    return args, kwargs

show(1, 2, color="blue")   # ((1, 2), {"color": "blue"})
```

A `lambda` is a function written as one expression. Use it as a tiny key or callback. If the body needs statements, write a `def`.

```
names = ["priya", "Anu", "ben"]
ordered = sorted(names, key=lambda s: s.lower())
```

Scope

A name is looked up in the LEGB order: **L**ocal to the function, then **E**nclosing functions, then the module **G**lobal, then **B**uiltins such as `len` and `print`.

Assignment inside a function makes that name local to the whole function. To assign to a module name, declare `global`. To assign to a name in an enclosing function, declare `nonlocal`. Reading a global needs no declaration.

A closure is a function that remembers variables from the function that built it.

```
def make_adder(n):
    def add(x):
        return x + n
    return add

add5 = make_adder(5)
add5(2)     # 7
```

The loop-closure trap: every lambda in this list sees the same `i`, which is `2` after the loop.

```
funcs = []
for i in range(3):
    funcs.append(lambda: i)
# each funcs[k]() returns 2

funcs = []
for i in range(3):
    funcs.append(lambda i=i: i)
# default binds the current i: 0, 1, 2
```

8. Lists, tuples, dicts, and sets

A **list** is an ordered, mutable sequence. Indexing is fast. Appending at the end is fast on average. Inserting at the front is slow, because every later item moves. Searching with `in` walks the list.

```
nums = [3, 1, 2]
nums.append(4)
nums.sort()
last = nums.pop()
```

`sort` changes the list. `sorted(nums)` returns a new list and leaves the original. Both take `key` and `reverse`.

A **tuple** is an ordered, immutable sequence. Use it for a fixed record, such as a point `(x, y)`, and as a dict key when every item inside is hashable. Immutable means you cannot rebind a slot. If a slot holds a list, that list can still change.

A **dict** maps keys to values. Keys must be hashable: immutable values such as strings, numbers, and tuples of hashables. Lists cannot be keys. Lookup, insert, and delete are fast on average. Since Python 3.7, dicts keep insertion order.

```
user = {"name": "Priya", "score": 9}
user["city"] = "Pune"
score = user.get("score", 0)
name = user.pop("name")
```

`user["missing"]` raises `KeyError`. `user.get("missing")` returns `None` or a default you pass. Iterate keys with `for key in user`, pairs with `user.items()`.

A **set** holds unique hashable items and does not keep a useful order for your logic. It gives fast membership and the operations `|` union, `&` intersection, and `-` difference.

```
seen = set()
seen.add("a")
"a" in seen
```

A `frozenset` is an immutable set and can be a dict key.

What to pick

You need	Use
An ordered row you will edit	list
A fixed group of fields	tuple, or a dataclass once you have classes
Lookup by a key	dict
Unique items and fast membership	set
A line at both ends	`collections.deque`
A count of each item	`collections.Counter`
A dict that fills a missing key	`collections.defaultdict`

9. Comprehensions and unpacking

A comprehension builds a list, set, or dict from a loop written as an expression. It is the ordinary way to transform a sequence.

```
squares = [n * n for n in range(5)]
evens = [n for n in range(10) if n % 2 == 0]
index = {name: i for i, name in enumerate(names)}
unique_sizes = {len(word) for word in words}
```

`enumerate` pairs each item with its index. `zip` walks several sequences together and stops at the shortest.

```
for i, name in enumerate(["a", "b"]):
    print(i, name)
pairs = list(zip([1, 2], ["a", "b"]))
```

Unpacking assigns several names at once.

```
x, y = (1, 2)
first, *rest = [1, 2, 3, 4]
a, b = b, a
```

The last line swaps without a temporary variable. `*rest` collects whatever remains into a list. You can unpack into a call: `print(*nums)` passes each item as its own argument, and `f(**options)` passes a dict as keyword arguments.

10. Files and paths

Use `pathlib.Path` for file names. It joins pieces with `/` in code, on every operating system.

```
from pathlib import Path
folder = Path("notes")
folder.mkdir(exist_ok=True)
path = folder / "today.txt"
path.write_text("hello\n", encoding="utf-8")
text = path.read_text(encoding="utf-8")
```

`open` is the lower-level tool. Always pass `encoding` for text. A `with` block closes the file even if an error happens.

```
with open(path, encoding="utf-8") as handle:
    for line in handle:
        print(line.rstrip("\n"))
```

Modes: `"r"` read text, `"w"` write text and truncate, `"a"` append, `"rb"` and `"wb"` for bytes. `"w"` destroys existing contents when the file opens. Read bytes when the file is an image, a zip, or anything that is not text.

JSON is the usual text format for structured data.

```
import json
payload = json.dumps({"ok": True})
data = json.loads(payload)
path.write_text(json.dumps(data), encoding="utf-8")
```

`json` understands dicts, lists, strings, numbers, Booleans, and `None`. It does not understand a custom class unless you convert that class to those types first.

11. Errors

An exception is how Python reports that a statement could not finish. The traceback shows the call stack, newest call last. Read it from the bottom: the exception type, the message, and the line that raised it. Then walk upward to see who called that line.

```
try:
    value = int(text)
except ValueError:
    value = None
else:
    print("parsed", value)
finally:
    print("done")
```

`except` runs when that exception type is raised. `else` runs when the `try` body did not raise. `finally` runs either way, including after `return`. Use `finally` or, better, `with`, to release a resource.

Catch the narrowest type that you can handle. A bare `except:` also catches `KeyboardInterrupt` and `SystemExit`, which inherit from `BaseException` rather than `Exception`. Catch `Exception` only at a boundary where you log and continue or exit cleanly.

Raise with `raise ValueError("amount must be positive")`. Chain causes with `raise RuntimeError("could not save") from error` so the original traceback stays attached. `raise` with nothing, inside an `except`, re-raises the current exception.

Define your own type when callers should be able to catch that case on purpose.

```
class Full(Exception):
    pass

def park():
    raise Full("no spots")
```

Python style prefers EAFP, “easier to ask forgiveness than permission”: try the operation and handle the exception. The other style, LBYL, checks first (“look before you leap”). A check-then-act pair can still fail if the world changes between the check and the act. Catching `KeyError` or `FileNotFoundError` is often the honest version.

12. Classes

A class is a blueprint. An instance is one built object. `self` is the instance the method was called on. You list it as the first parameter. You do not pass it at the call: `ticket.leave()` passes `ticket` as `self`.

```
class Ticket:
    def __init__(self, plate, entry):
        self.plate = plate
        self.entry = entry
        self.exit = None

    def close(self, now):
        self.exit = now
        return now - self.entry

ticket = Ticket("KA01", 10)
ticket.close(25)
```

`__init__` initializes a new instance. It does not return the instance. Python creates the instance, then calls `__init__`.

Attributes set on `self` belong to that instance. Attributes set on the class are shared.

```
class Lot:
    city = "Pune"          # shared
    def __init__(self):
        self.spots = []    # per instance
```

A mutable class attribute, such as a list defined on the class, is one list for every instance that does not override it. Put per-instance containers on `self` inside `__init__`.

Methods

An ordinary method receives the instance. A `@classmethod` receives the class, and is how you write an alternate constructor. A `@staticmethod` receives neither. It is a function parked in the class because it belongs to that topic. Prefer a module-level function when it does not need the class at all.

```
class Link:
    def __init__(self, code, url):
        self.code = code
        self.url = url

    @classmethod
    def from_pair(cls, pair):
        code, url = pair
        return cls(code, url)
```

A `@property` looks like an attribute and runs a method. Use it when reading a value should stay cheap and stable, and when replacing a stored attribute with a calculation should not break callers.

```
class Circle:
    def __init__(self, radius):
        self.radius = radius

    @property
    def area(self):
        return 3.14159 * self.radius ** 2
```

Inheritance

A subclass is a more specific blueprint. It reuses the superclass and can override methods. `super()` calls the next implementation in the method resolution order.

```
class Vehicle:
    def spots(self):
        return 1

class Bus(Vehicle):
    def spots(self):
        return 5
```

`isinstance(bus, Vehicle)` is true. `type(bus) is Vehicle` is false. Prefer `isinstance` when you mean "can this be treated as."

The **MRO** is the order Python searches parent classes. With single inheritance it is the obvious chain. With multiple inheritance, Python uses the C3 order so each parent appears before its own parents and so a class is not visited twice. You rarely need multiple inheritance. When you do, read `Class.__mro__` before you trust `super()`.

A leading underscore, `_cache`, is a convention: "internal, callers should use the public method." Two leading underscores, `__cache`, trigger name mangling to `_ClassName__cache`, which avoids accidental overrides in subclasses. It is not a security boundary. Nothing in a Python object is hidden from a determined caller.

Dataclasses and enums

A dataclass writes `__init__`, `__repr__`, and `__eq__` for a class that mostly stores fields.

```
from dataclasses import dataclass

@dataclass
class Point:
    x: float
    y: float

    def moved(self, dx, dy):
        return Point(self.x + dx, self.y + dy)
```

`frozen=True` makes assignment to fields fail, which is what you want for a value that should be a dict key. `slots=True` on a dataclass stores fields more tightly and rejects accidental new attributes.

An enum is a fixed set of named constants.

```
from enum import Enum

class State(Enum):
    HELD = "held"
    SOLD = "sold"
```

Compare with `is` or `==` against the member, not against the raw string scattered through the program.

13. The data model

Python's syntax is sugar over methods with double underscores. When you write `len(box)`, Python calls `box.__len__()`. When you write `a + b`, it calls `a.__add__(b)`. When you write `box[0]`, it calls `box.__getitem__(0)`. Implementing these methods is how your class joins the language.

You write	Python calls
`len(obj)`	`__len__`
`obj[key]`	`__getitem__`
`for x in obj`	`__iter__`
`str(obj)` and print	`__str__`
the prompt and containers	`__repr__`
`obj == other`	`__eq__`
`hash(obj)` and dict keys	`__hash__`
`with obj`	`__enter__` and `__exit__`
`obj()`	`__call__`

`__repr__` should be unambiguous, ideally a string that shows how to rebuild the value. `__str__` can be friendly. If you define only `__repr__`, `print` uses it too.

If you define `__eq__`, Python sets `__hash__` to `None` unless you define that too, and the objects cannot be set members or dict keys. Equal objects must have equal hashes. The hash of a mutable object is a trap, because changing the object after inserting it into a set loses the entry. Hash immutable values.

Duck typing means "if it has the methods you need, use it." A function that calls `write` can accept a file, a `StringIO`, or your own class with `write`. A `Protocol` in the typing chapter names that shape for the type checker. An abstract base class, from `abc`, refuses to instantiate a subclass that forgot a required method. Use a Protocol when you want structure. Use an ABC when you want a shared base that enforces a contract.

14. Iterators, generators, and with

An **iterable** can produce an iterator, usually through `__iter__`. An **iterator** produces the next value through `__next__` and raises `StopIteration` when it is done. A `for` loop gets an iterator and calls next until that exception. You rarely write the class form. You write a generator.

A **generator** is a function that uses `yield`. Calling it does not run the body. It returns a generator object. Each `next` runs until the next `yield`, pauses, and resumes there later. Generators produce one item at a time, so a billion-row scan does not become a billion-row list.

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1

for value in countdown(3):
    print(value)
```

`yield from other` forwards each item from another iterable. A generator expression is the lazy cousin of a list comprehension: `(n * n for n in range(10))`. It does not compute the squares until something walks it. Walking it twice does nothing the second time. It is spent.

`itertools` holds the small tools: `chain`, `islice`, `groupby`, `count`, `cycle`. Reach for it before you write a nested loop that only reorganizes items.

Context managers

`with` guarantees cleanup. The object's `__enter__` runs at the start. `__exit__` runs at the end, whether the block succeeded, returned, or raised. Files, locks, and database transactions are the usual examples.

`contextlib.contextmanager` lets you write one as a generator: the code before `yield` is enter, the code after is exit.

```
from contextlib import contextmanager

@contextmanager
def section(title):
    print("start", title)
    try:
        yield
    finally:
        print("end", title)
```

15. Decorators

A decorator is a function that takes a function and returns a function, applied with `@`. The two spellings below are the same.

```
def shout(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs).upper()
    return wrapper

@shout
def greet(name):
    return f"hello {name}"
```

Use `functools.wraps` so the wrapper keeps the original name and docstring.

```
import functools

def shout(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        return str(func(*args, **kwargs)).upper()
    return wrapper
```

A decorator can take arguments by adding one more nested function: the outer function receives the arguments and returns the real decorator. Built-in decorators you will use constantly are `@property`, `@classmethod`, `@staticmethod`, `@dataclass`, and `@functools.lru_cache` or `@functools.cache` for a pure function you want to memoize. Cache only functions whose results depend only on their arguments. A cached function that reads a global or the clock will lie.

16. Modules, packages, and installs

A **module** is a `.py` file. A **package** is a folder of modules. `import math` loads the module and binds the name `math`. `from math import sqrt` binds `sqrt` itself. `import math as m` renames it locally.

Importing runs the module's top level once and caches the module in `sys.modules`. A second import does not run it again. Keep the top level boring: definitions, not a program that starts work. Put the program under this guard so importing the file does not launch it:

```
def main():
    print("running")

if __name__ == "__main__":
    main()
```

`__name__` is `"__main__"` only when that file was the one you started. When another file imports it, `__name__` is the module's name.

A package folder may contain `__init__.py`. That file runs when the package is imported. Relative imports, `from . import codec`, stay inside the package. They work when the package is imported as a package, and they fail if you run an inner file as a loose script. Run `python -m package.module` instead.

The import path is `sys.path`: the script's directory, then installed locations. Do not modify it in ordinary programs. Name your modules so they do not shadow the standard library. A file called `json.py` will hide the real `json`.

Environments and packages

A virtual environment is a private folder of installed libraries for one project, so project A's library version does not change project B.

```
python -m venv .venv
.venv\Scripts\activate
python -m pip install requests
```

`pip` installs packages from the Python Package Index and from local folders. Pin versions in a requirements file or, better for a real project, declare them in `pyproject.toml`. That file is also where a tool reads the project's name, the Python version it needs, and the build backend. You do not need to publish a package to benefit from a venv and a pinned list. You need them the moment a program must run again next month.

The standard library ships with Python. Third-party packages do not. Prefer the standard library when it does the job.

17. Type hints

Hints describe the kinds of values you intend. Python does not enforce them while the program runs. A type checker such as `pyright` or `mypy` reads them as a separate step. Hints pay off in anything larger than a script: editors complete names, and a wrong argument is caught before the program runs.

```
def clamp(n: int, low: int, high: int) -> int:
    if n < low:
        return low
    if n > high:
        return high
    return n

def find(code: str) -> str | None:
    return codes.get(code)
```

`str | None` means a string or `None`. `list[str]`, `dict[str, int]`, and `tuple[int, int]` name the insides. `tuple[int, ...]` is a tuple of any length of ints. A bare `list` hint does not say what the list holds.

```
from collections.abc import Callable, Iterable, Sequence

def total(nums: Iterable[int]) -> int:
    return sum(nums)

def apply(fn: Callable[[int], int], n: int) -> int:
    return fn(n)
```

Hint the most general thing you actually use. If you only iterate, hint `Iterable`. If you index and take `len`, hint `Sequence`.

A `Protocol` names a structural shape. Any object with those methods satisfies it, with or without a shared base class.

```
from typing import Protocol

class Writer(Protocol):
    def write(self, text: str) -> int: ...
```

A `TypedDict` describes a dict with known keys, which is what JSON objects become. A dataclass is still the better choice when you own the type. Use `TypedDict` at the edge where the data really is a dict.

Generics use `TypeVar` or, in current Python, a type parameter: `class Box[T]`. You need them when a function's result type depends on its input type, such as "this cache stores whatever type you put in."

Casts and `Any` switch the checker off for that expression. Use them at a boundary you do not control, and keep them small. A hint of `Any` on every function is the same as no hints.

18. The standard library you actually use

Learn these modules by the job they do. The full library is larger. This is the set you should be able to reach for without searching.

Job	Tool
Paths and small file reads	`pathlib.Path`
JSON text	`json`
CSV tables	`csv`
Dates and times	`datetime`, with timezone-aware values
Counts, default dicts, double-ended queues, named tuples	`collections`
Memoize, partial application, reduce, wraps	`functools`
Iterator helpers	`itertools`
Patterns in text	`re`
Logs	`logging`
Command-line flags	`argparse`
Fixed constants	`enum`
Shallow and deep copies	`copy`
Math on numbers	`math`
Simulations and shuffles	`random`
Tokens, passwords, secrets	`secrets`
Environment variables	`os.environ`
Running a process	`subprocess`
Exact decimals	`decimal`
In-memory text files for tests	`io.StringIO`

datetime

A **naive** datetime has no timezone. An **aware** datetime has one. Mix them and Python raises, which is the right failure. Store instants as aware UTC. Convert to a local zone only for display.

```
from datetime import datetime, timezone
now = datetime.now(timezone.utc)
stamp = now.isoformat()
```

`date` is a calendar day. `timedelta` is a duration. Subtracting two datetimes returns a timedelta.

collections

```
from collections import Counter, defaultdict, deque

counts = Counter(["a", "b", "a"])
counts.most_common(1)

groups = defaultdict(list)
groups["priya"].append(1)

line = deque()
line.append(1)
line.appendleft(0)
line.pop()
```

re

A regular expression is a tiny language for text shapes. `re.search` finds a match anywhere. `re.fullmatch` requires the whole string to match. `re.findall` returns the matched pieces. Compile a pattern you reuse. Prefer string methods when the job is a fixed prefix, suffix, or split. Reach for `re` when the shape varies: an email-like token, a repeated number, optional pieces.

```
import re
match = re.fullmatch(r"[A-Z]{2}\d{2}", "KA01")
code = match.group(0) if match else None
```

logging

Levels, from quieter to louder: DEBUG, INFO, WARNING, ERROR, CRITICAL. Configure once at program start. Get a logger per module with `logging.getLogger(__name__)`. Log an event with fields, not a paragraph you will regret parsing.

```
import logging
log = logging.getLogger(__name__)
log.info("opened code=%s", code)
```

The `%s` style lets the logger skip the formatting work when the level is disabled. Pass exceptions with `log.exception("save failed")` inside an `except` block so the traceback is recorded.

argparse and subprocess

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("path")
parser.add_argument("--limit", type=int, default=10)
args = parser.parse_args()
```

`subprocess.run(["python", "tool.py"], check=True)` runs a program and raises if it exits with an error. Pass arguments as a list. A single string with `shell=True` hands that string to a shell, which is how untrusted text becomes a command. Keep `shell=False` unless you have a specific reason and a trusted command.

random and secrets

`random` is for games, samples, and tests. It is not for passwords or tokens. `secrets.token_urlsafe(16)` and `secrets.choice` draw from the operating system's cryptographic source. Seeds exist so a simulation can be repeated: `random.seed(0)`.

19. Concurrency

Concurrency means the program has more than one thing in progress. It does not always mean more than one CPU core is running your Python at the same instant.

The GIL

CPython, the usual interpreter, has a global interpreter lock. One thread at a time runs Python bytecode. Threads still help when the work waits on the network, a disk, or a database, because those waits release the lock and another thread can run. Threads do not speed up a tight loop of pure Python arithmetic. For that, use separate processes, or a library that does the heavy work in native code and releases the lock.

Threads

A `Thread` runs a function beside the caller. Share objects only with a plan. A lock (`threading.Lock`) lets one thread into a block at a time. Acquire locks in a fixed order so two threads cannot each hold the lock the other wants. Prefer handing each thread its own data and collecting results through a `queue.Queue`.

```
from threading import Thread

def work(name, out):
    out.append(name.upper())

box = []
thread = Thread(target=work, args=("priya", box))
thread.start()
thread.join()
```

The list append here is one bytecode operation and is safe in CPython. A read-modify-write such as `n = n + 1` on a shared `n` is not safe. Use a lock or a `queue`.

Processes

`multiprocessing` and `concurrent.futures.ProcessPoolExecutor` run functions in other interpreters, each with its own GIL. Arguments and return values are pickled, which means copied across a boundary. Processes pay startup cost and cannot share an ordinary list. Use them for CPU-heavy work that splits into independent chunks.

```
from concurrent.futures import ProcessPoolExecutor

def square(n):
    return n * n

if __name__ == "__main__":
    with ProcessPoolExecutor() as pool:
        print(list(pool.map(square, [1, 2, 3])))
```

The `__main__` guard is required on Windows. The child process imports the module, and without the guard it would spawn more children forever.

asyncio

`async` and `await` are cooperative concurrency on one thread. A task runs until it awaits something, usually I/O. The event loop then runs another task. Nothing preempts you. A CPU loop or a blocking `time.sleep` inside an async function freezes every task. Use `asyncio.sleep`, async libraries, and `asyncio.to_thread` for a blocking call you cannot avoid.

```
import asyncio

async def fetch(name):
    await asyncio.sleep(0.1)
    return name.upper()

async def main():
    results = await asyncio.gather(fetch("a"), fetch("b"))
    return results

asyncio.run(main())
```

`gather` starts the tasks together and waits for all of them. One event loop per thread is the rule. `asyncio.run` creates the loop, runs the coroutine, and closes the loop.

Choose by the bottleneck. Waiting on many sockets: asyncio or threads. Burning CPU in Python: processes. A single straightforward script: neither.

20. Tests, debugging, and style

Tests

A test calls a function with a fixed input and checks the output. `pytest` is the tool most projects use. A test is a function whose name starts with `test_`, and a failing `assert` is a failing test.

```
def clamp(n, low, high):
    return max(low, min(high, n))

def test_clamp_low():
    assert clamp(-1, 0, 10) == 0

def test_clamp_inside():
    assert clamp(4, 0, 10) == 4
```

Run `python -m pytest`. Test the edges: empty input, one item, the boundary value, the error case. For exceptions:

```
import pytest

def test_rejects_blank():
    with pytest.raises(ValueError):
        parse(" ")
```

The standard library's `unittest` does the same job with classes and `self.assertEqual`. Knowing either is enough. Mock only the edges you do not own: the clock, the network, a random source. A test that mocks the function it claims to test proves nothing.

Give tests a fake clock and a fake repository, the same idea as the system-design book. Pass them in. Do not make the function reach for a hidden global that the test cannot replace.

Debugging

Read the traceback first. Then print the value you did not expect, or drop into the debugger with `breakpoint()`. From there, `n` steps to the next line, `s` steps into a call, `c` continues, `p expr` prints an expression. `repr(value)` shows a value more honestly than `print` when spaces and types matter.

Bisect a mystery: comment half the path, or return early, until the surprise disappears. The last piece you removed is where to look.

Style

PEP 8 is the community style. The parts that matter daily:

- Four spaces per indent. Lines around 88 or 99 characters, not a hard religion.
- Functions and variables in `snake_case`. Classes in `CapWords`. Constants in `UPPER_SNAKE`.
- Two blank lines between top-level functions and classes.
- Imports in three groups: standard library, third party, your package. One import per line for modules. Avoid `from module import *`.
- Names that say what the value is. `elapsed_seconds` beats `t` once a function is longer than a few lines.
- A function does one job. If you describe it with "and," consider splitting it.

- Raise specific exceptions. Return `None` only when absence is a normal answer, and hint `| None`.

Formatters such as `ruff format` or `black` end arguments about whitespace. Linters such as `ruff` catch unused names, shadowing, and mutable defaults. Run them. They are part of knowing the language in a team.

21. Pitfalls

These are the mistakes that happen after the syntax is familiar.

- **Shared mutable default.** `def f(items=[])` uses one list forever. Use `None` and create the list inside.
- **Alias versus copy.** `b = a` shares a mutable object. Copy when you mean a separate value.
- **Late closure.** Lambdas in a loop share the loop variable. Bind it with a default argument.
- **`is` for values.** Use `is` for `None`. Use `==` for everything you mean as a value. Small integers may share objects by accident. Do not depend on that.
- **Float money.** Use `Decimal`.
- **Mutable dict keys.** Only hashables. Do not change an object that is already in a set.
- **Modifying a list while looping over it.** Iterate over a copy, or build a new list.
- **Except that is too wide,** then a bug disappears into the handler. Catch the type you expect.
- **Bare open without `with`** and without an encoding.
- **Naive and aware datetimes** mixed in one expression.
- **Shadowing a builtin.** Naming a variable `list` or `id` hides the builtin in that scope.
- **A module named like the standard library,** such as `random.py` in your folder.
- **Forgetting the main guard** before using `multiprocessing` on Windows.
- **Blocking inside `async def`.** The whole loop waits with you.
- **Caching a function that is not pure.**
- **Comparing floats with `==`** after arithmetic. Compare with a tolerance, or use decimals.
- **Assuming dict order did not exist.** It does, since 3.7. Still do not use a dict as a priority queue. Use a heap (`heapq`) when you always need the smallest item.

```
import heapq
heap = []
heapq.heappush(heap, (2, "b"))
heapq.heappush(heap, (1, "a"))
first = heapq.heappop(heap)   # (1, "a")
```

22. Glossary and a study plan

Term	Meaning
Object	A value with a type and an identity
Name	A label pointing at an object. Assignment moves the label
Immutable	The object's value cannot change. int, str, tuple
Mutable	The object can change in place. list, dict, set, most instances
Identity	Whether two names are the same object. Test with `is`
Iterable	Something a `for` loop can walk
Iterator	The walker. Produces the next item or stops
Generator	A function that yields items and pauses between them
Scope	The region where a name is visible. LEGB
Closure	A function that keeps using variables from where it was created
Exception	An object raised to report that a statement could not finish
Instance	One object built from a class
Method	A function defined on a class, receiving the instance as `self`
MRO	The order parent classes are searched
Decorator	A function that wraps another function
Context manager	An object that `with` uses to set up and clean up
Module	One `.py` file that can be imported
Package	A folder of modules
Hint	A note about types, checked by a type checker, not by Python itself
GIL	The lock that lets one thread run Python bytecode at a time
Awaitable	An object you can `await`, usually a coroutine
Hashable	An object that can be a dict key or a set member. It needs a stable hash

What "I know Python" means

You can write a small program that reads a file, computes with dicts and lists, defines a function and a class, handles a bad input with an exception, and tests the function. You can explain why two names saw the same list change. You can say when you want a thread, a process, or `async`. You can read a traceback. That is the language. Libraries for the web, data, or machine learning sit on top of this map. They are vocabularies of functions. They are not a second language.

Ten sessions

1. Names, mutability, `is` and `==`. Draw boxes and arrows for three tiny programs.
2. Strings, f-strings, slices, and a file read with `pathlib`.
3. Lists, dicts, sets, comprehensions. Solve "count the words in a file."

4. Functions, defaults, `*args`, scope, and the closure trap.
5. Exceptions. Turn the word count into a function that raises on a missing file and returns a `Counter`.
6. Classes, dataclass, property, inheritance. Model a ticket with `close`.
7. Generators and `with`. Stream the word count so the whole file is not one string if you can avoid it. Line by line already does this.
8. A decorator that times a function, using `wraps`. Add `lru_cache` to a pure function and show a second call is instant.
9. Type hints on the word counter. Run a type checker if one is installed.
10. One test file with pytest. One threaded or async example that waits, and one sentence on why a CPU loop would not speed up under threads.

After those ten, pick a project you care about and keep this PDF closed until a concept fails you. Then open that chapter only. The concepts are the ones in the contents. Knowing them is predicting the next line, not recognizing the heading.

End of the offline Python guide.

23. Interview questions

These are the questions interviewers use to see whether the concepts in this book are available when you speak. Each one has the point of the question and an answer you can say aloud. A strong answer is short, then one example, then the trap if there is one. You do not need to recite the paragraph.

Names and objects

What is the difference between `is` and `==`? `==` asks whether the values compare equal. `is` asks whether both names point at the same object. Use `is` for `None`. Two lists with the same items are `==` and are not `is`.

Are arguments passed by value or by reference? Neither, in the C++ sense. The parameter name is stuck on the same object the caller passed. Reassigning the parameter moves only the local label. Mutating the object, such as `append`, is visible to the caller because there is one object.

What is mutable and what is immutable? Immutable: `int`, `float`, `str`, `tuple`, `frozenset`, `bytes`. Mutable: `list`, `dict`, `set`, `bytearray`, and most class instances. Immutability is why strings and tuples can be dict keys and why "changing" an int actually binds a new object.

Why did changing one list change another? Both names were labels on one list. Assignment does not copy. A shallow copy copies the outer list and still shares the inner objects. `copy.deepcopy` copies the inners too.

What does `a, b = b, a` do? The right side builds a tuple of the current values, then the left side unpacks it. No temporary variable is required.

Types and text

Why is `0.1 + 0.2` not `0.3`? Floats are binary fractions with limited bits. Those decimals are not exact in binary. For money, use `decimal.Decimal` constructed from strings.

What is the difference between `str` and `bytes`? `str` is Unicode text. `bytes` is raw octets. Convert with `encode` and `decode`, usually UTF-8. You cannot mix them with `+`.

How do you join many strings? `" ".join(parts)`. Repeated `+=` in a loop builds many intermediate strings. Join walks the pieces once.

What is an f-string? A string literal with expressions in braces, evaluated when that line runs: `f"{name} scored {score}"`.

Which values are false? `None`, `False`, numeric zero, and empty containers and strings. An empty list is false and is not `None`. Test absence with `is None`.

Collections

When do you pick a list, a tuple, a dict, or a set? A list is an ordered collection you edit. A tuple is a fixed record. A dict is a lookup by key. A set is unique items with fast membership. Dict keys and set items must be hashable.

Why can a tuple be a dict key while a list cannot? A key's hash must stay stable. A list can change after insertion, so it is unhashable. A tuple is hashable only when every item inside it is hashable.

Are dicts ordered? Yes. Since Python 3.7, insertion order is part of the language. That does not make a dict a priority queue. Use `heapq` when you always need the smallest item.

What is the cost of `in`? Average constant time for a dict or a set. Linear for a list or a tuple, because those are scanned.

What do `append` and `insert(0, x)` cost? Append is amortized constant time. Insert at the front moves every later item, so it is linear. A queue that grows at both ends wants `collections.deque`.

`list` versus tuple in an interview sentence. Same indexing and slicing. A list can grow and change slots. A tuple cannot, so it can be a key and it signals a fixed shape.

What does `sorted` return, and what does `list.sort` return? `sorted` returns a new list. `list.sort` sorts in place and returns `None`. Both take `key` and `reverse`. Python's sort is stable: equal keys keep their original order.

Control flow and functions

What is a default-argument bug? Defaults are created once, at definition. `def add(item, bucket=[])` shares one list across calls. Use `bucket=None` and create a new list inside the function.

What are `*args` and `**kwargs`? `*args` collects extra positional arguments into a tuple. `**kwargs` collects extra keyword arguments into a dict. In a call, `*seq` and `**mapping` unpack outward.

What is LEGB? The lookup order for a name: local, enclosing function, global module, builtin. Assignment inside a function makes the name local for the whole function. `global` and `nonlocal` are how you assign further out.

What is a closure? A function that uses variables from the function that created it, after that outer call has returned. In a loop, every lambda sees the final value of the loop variable unless you bind the current value as a default: `lambda i=i: i`.

What is the `else` on a `for` loop? It runs when the loop ended without `break`. It is the "not found" branch of a search.

What does `//` do? Floor division. It divides and rounds toward negative infinity, which is not the same as truncating toward zero for negative numbers.

Classes

What is `self`? The instance the method was called on. You declare it as the first parameter. The call `obj.method()` passes `obj` in for you.

`@classmethod` versus `@staticmethod` versus a normal method. A normal method receives the instance. A classmethod receives the class, which is how alternate constructors stay correct for subclasses. A staticmethod receives neither. If it does not need the class, a module-level function is often clearer.

What is `@property`? A method that callers read like an attribute. Use it when a stored field later becomes a calculation and you do not want to change every caller.

`__str__` versus `__repr__`. `__repr__` is the unambiguous developer form, ideally enough to see how the value was built. `__str__` is the friendly form `print` prefers. If only `__repr__` exists, `print` uses it.

`__init__` versus `__new__`. `__new__` creates the instance and is the place to intervene for immutables such as `tuple` subclasses. `__init__` initializes an instance that already exists. Almost every class you write only needs `__init__`.

What is the MRO? The method resolution order: the sequence of classes Python searches for an attribute. Single inheritance is a straight line. Multiple inheritance uses the C3 order. Read `Class.__mro__` when `super()` surprises you.

What does a leading underscore mean? `_name` means internal by convention. `__name` is mangled to `_Class__name` so a subclass does not override it by accident. Neither one is a security barrier.

Why a dataclass? The class mostly stores fields. The decorator writes `__init__`, `__repr__`, and `__eq__`. `frozen=True` makes instances hashable if the fields are hashable.

What is duck typing? If an object has the methods you call, it is acceptable. You do not require a particular base class. A `Protocol` is that idea written down for a type checker.

Errors, files, and `with`

What runs in `try`, `except`, `else`, and `finally`? `try` is the risky work. `except` handles a matching exception. `else` runs only when `try` did not raise. `finally` runs in every case, including after a `return`, and is where you release a resource if you are not using `with`.

Why avoid a bare `except:`? It also catches `KeyboardInterrupt` and `SystemExit`, which inherit from `BaseException`, not `Exception`. Catch the type you can handle.

What is EAFP? Easier to ask forgiveness than permission. Try the operation and catch `KeyError` or `FileNotFoundError`. A check followed by the action can still fail if something changes in between.

What does `with` guarantee? `__enter__` runs at the start and `__exit__` runs when the block ends, including when it raises or returns. That is why files are opened with `with`. Always pass `encoding` for text.

What is `if __name__ == "__main__"`? That block runs only when the file is started directly. Importing the file sets `__name__` to the module name, so the program does not start as a side effect of the import. On Windows, `multiprocessing` needs this guard.

Iterators, generators, and decorators

Iterable versus iterator. An iterable can produce an iterator, which a `for` loop does for you. An iterator remembers its position and returns the next item until it raises `StopIteration`. A list is iterable. Calling `iter` on it gives an iterator.

Why a generator? `yield` produces one item at a time and pauses. A large file or a long range does not become one giant list. Calling the generator function does not run the body. It returns the generator. A generator expression is spent after one walk.

List comprehension versus generator expression. Square brackets build the whole list now. Parentheses build a generator that computes on demand. Use the list when you need length, indexing, or several walks.

What is a decorator? A function that takes a function and returns a function. `@deco` above `def` is the same as `fn = deco(fn)` after the definition. Use `functools.wraps` so the wrapper keeps the original name and docstring.

What does `functools.lru_cache` assume? The function is pure: the same arguments always mean the same result, and the work is worth storing. A cached function that reads the clock or a global returns a stale answer.

Modules and types

What is a virtual environment? A private folder of third-party packages for one project, so two projects can depend on different versions. Create it with `python -m venv .venv` and install with the `pip` inside it.

Do type hints change runtime behavior? No. The interpreter does not enforce them. A type checker such as pyright or mypy reads them as a separate step. `str | None` means a string or `None`.

What is a `Protocol`? A structural type. Any object with the listed methods matches, even with no shared base class. That is duck typing for the checker.

Concurrency

What is the GIL? In CPython, a lock that allows one thread to run Python bytecode at a time. Threads still help for waiting on network or disk, because that waiting releases the lock. A pure Python CPU loop does not get faster with more threads. Use processes, or native code that releases the lock, for CPU work.

Thread, process, or asyncio? Threads for blocking I/O with ordinary libraries. Processes for CPU-bound Python. Asyncio for many concurrent sockets in one thread, using libraries that await. A blocking call inside `async def` freezes the loop.

Is `n = n + 1` safe across threads? No. Two threads can read the same `n` and both write the same next value. Use a lock, or do not share the variable. List `append` of one item is a different, simpler operation. Do not generalize from it.

What is a race? Two threads use the same data, the result depends on the order they run, and that order is not fixed. The fix is not to share, or to make the critical update atomic with a lock or a queue.

Short coding questions

Say the idea first, then write it. Handle the empty input aloud so the interviewer hears that you thought about it.

Count words in a file, case-insensitive. Open with `pathlib` or `with`, walk lines, `split`, and add to a `Counter` after `lower`. Do not read a huge file as one string if line-by-line works.

```
from collections import Counter
from pathlib import Path

def word_counts(path):
    counts = Counter()
    text = Path(path).read_text(encoding="utf-8")
    for word in text.lower().split():
        counts[word] += 1
    return counts
```

Find the first repeated character. Walk the string with a set of characters already seen. The first character already in the set is the answer. If the loop ends, there is no repeat. This is linear time and uses memory proportional to the alphabet you actually saw.

```
def first_repeat(text):
    seen = set()
    for char in text:
        if char in seen:
            return char
        seen.add(char)
    return None
```

Group words that are anagrams. The key is the letters in sorted order. A dict of lists collects words that share a key.

```
def group_anagrams(words):
    groups = {}
    for word in words:
        key = "".join(sorted(word))
        groups.setdefault(key, []).append(word)
    return list(groups.values())
```

Two numbers in a list that add to a target. One pass, a dict from value already seen to its index. For each new number, ask whether `target - number` is already in the dict. Average linear time. Say what you return if no pair exists.

```
def two_sum(nums, target):
    seen = {}
    for index, num in enumerate(nums):
        need = target - num
        if need in seen:
            return seen[need], index
        seen[num] = index
    return None
```

Flatten one level of nested lists. A double comprehension or a loop that extends. Say whether deeper nesting is in scope. One level is `[item for row in rows for item in row]`.

Remove duplicates and keep the first-seen order. A dict preserves insertion order. `list(dict.fromkeys(items))` does it. A set would lose order.

Write a timer decorator. Take the function, return a wrapper that uses `*args, **kwargs`, records time around the call, and returns the original result. Apply `functools.wraps`.

```
import functools
import time

def timed(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
            elapsed = time.perf_counter() - start
            print(f"{func.__name__} took {elapsed:.4f}s")
    return wrapper
```

Implement a tiny LRU in words. A dict plus order. On get, move the key to the most recently used end. On put past capacity, drop the least recently used key. The book's system-design volume has a full

class. In a Python interview, describe those two structures and the two operations before you write the nodes.

Questions you can ask back

When they invite questions, pick one that shows you care how the code is maintained: how they test, how they pin dependencies, and whether CPU work and I/O work are separated in the service you would join. You are not proving you read a list. You are showing you know which concepts touch the job.

How to practice

1. Cover the answer. Say yours out loud in under a minute.
2. Uncover and keep only the sentence you missed.
3. For each coding question, write it in a file and run it, including the empty input and a one-item input.
4. Stop when you can answer the GIL, the mutable default, `is` versus `==`, a generator, and a decorator without looking. Those five separate people who have used Python from people who have read a list of keywords.

End of the interview questions.